

数据备份软件 · 组内开工说明

这份文档的目的是让三个人对同一件事有同一张图：项目要交什么、你负责哪块、你的代码要满足什么契约、什么时候交。读完请按最后一节逐条回复确认。

v0.1 2026-08-31 待组内确认 Windows · C++17 · Python

01 我们在做什么，以及分数怎么来

做一款 Windows 上的数据备份软件：把一棵目录树完整地存成一个文件，之后能一字不差地还原回去——包括文件本身，也包括链接关系、时间戳和权限。

课程基础分 40 分（备份 + 还原），扩展项各自计分。我们选了六个扩展，理论上限如下：

模块	分值	负责人	说明
基础备份还原	40	Z	目录遍历、容器读写、还原编排
文件类型支持	10	Z	符号链接、硬链接、目录联接
元数据支持	10	Z	属性位、三时间戳、属主 SID、权限表
打包解包	10 × 算法数	A	做两种算法就是 20 分
压缩解压	10 × 算法数	A	同上
加密解密	10 × 算法数	B	同上
图形界面	10	B	Python + PySide6

注意这里的乘法。最终小组得分 = 项目完成分 × 项目难度分 ÷ 100。也就是说难度分（上表之和）是个**乘数**，不是加数。文档写得好、代码规范、答辩顺利拿到的"完成分"，会被这个乘数整体放大或缩小——所以谁都不能只顾自己那一块跑分，文档和代码质量是三个人共同的乘数。

两条红线

这两条写在课件里，踩了就是直接扣分，而且是那种事后补不回来的扣分。先看这两条，再看别的。

红线 1 后台逻辑不能用脚本语言

开发语言选择：C/C++/C#/Java/Go；用户界面可以采用脚本语言编写；但若后台逻辑也选择脚本语言，则小组基础分记 10 分。

基础分满分 40。备份逻辑只要有一部分写进 Python，直接掉到 10 分，后面所有扩展做得再漂亮都补不回这 30 分。

所以：备份、还原、打包、压缩、加密，全部在 C++ 里。Python 只允许出现在界面层，而且界面层不做任何业务判断——它只负责收参数、起进程、画进度条。

红线 2 扩展功能不能"直接"用第三方库

对所有扩展功能，如使用第三方库/程序/代码"直接"实现，对应功能扩展分总分记为原来的 50%。

这一条主要砸在 A 和 B 头上：压缩不能调 zlib / miniz，加密不能调 OpenSSL / Windows CryptoAPI，打包不能调 libarchive。算法要自己写。

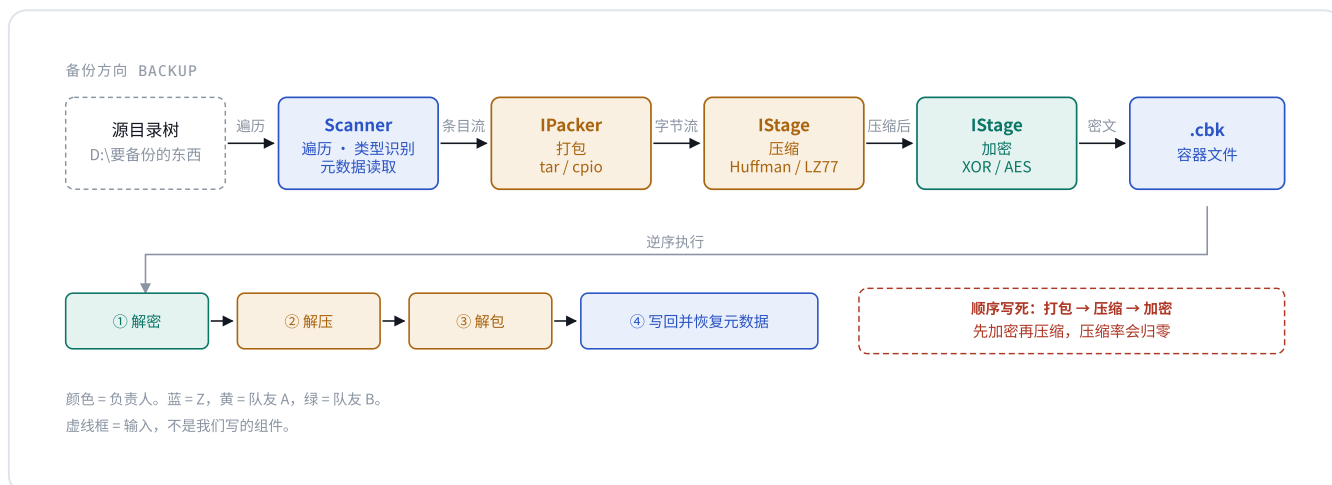
需要区分"替我实现了功能"和"工程基础设施"：

用法	判定	理由
<code>zlib</code> / <code>miniz</code> 做压缩	扣分	压缩功能本身被库替代了
<code>OpenSSL</code> / <code>CryptoAPI</code> 做 AES	扣分	同上
<code>libarchive</code> 做 tar	扣分	同上
<code>GoogleTest</code> 写单元测试	不扣	评分表反而鼓励"使用第三方测试工具"
<code>PySide6</code> 做界面	不扣	课件明确允许界面用脚本语言
<code>Win32 API</code> 读权限表	不扣	操作系统 API，不是第三方库

保守做法：核心库 `core/` 零第三方依赖，只用 Windows SDK 和 C++ 标准库。第三方依赖只准出现在 `tests/` 和 `gui/` 两个目录里。

整件事怎么串起来

备份产物是一个文件（后缀 `.cbk`），不是一个拷贝出来的目录。所有人的代码都是这条流水线上的一段。



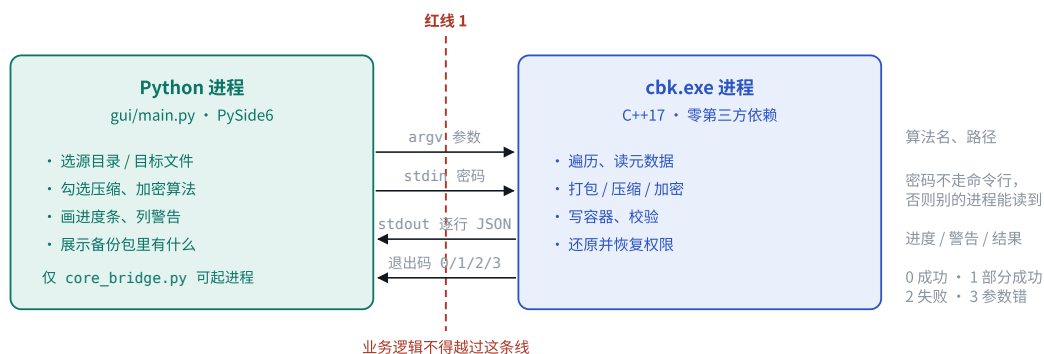
整条流水线只有一条路径。A 和 B 各自负责其中一段字节变换，两段都实现同一个 `IStage` 接口，谁在前谁在后由配置决定——但顺序被程序强制成“打包 → 压缩 → 加密”，配错会直接报参数错误退出。

为什么加密必须排在压缩后面

加密的输出是高熵的伪随机字节。对伪随机数据做压缩，压缩率不但归零，还会因为码表开销让文件变大。反过来先压后加，两者都正常工作。这不是偏好问题，是信息论问题，所以直接写死在代码里，不给配错的机会。

界面和核心是两个进程 B 重点

Python 界面不“调用”C++ 代码，它启动一个 C++ 程序。这是红线 1 在工程上的落地方式——两个进程之间隔着操作系统，业务逻辑不可能悄悄漏到 Python 那边去。



整个 Python 侧只有 `core_bridge.py` 一个文件被允许启动 `cbk.exe`。这条规矩不是洁癖：它保证将来换桥接方式时只改一个文件，也保证业务逻辑不会一点点渗进 Python 层。

B 需要知道的四件事

算法列表向核心要，别写死

启动时先跑一次 `cbk info`，核心会回一段 JSON 说明自己支持哪些打包、压缩、加密算法。界面拿这个填下拉框。A 以后新加一个压缩算法，界面一行都不用改就多出一个选项。

密码走标准输入

命令行参数在 Windows 上能被其它进程读到，也会进日志。所以传 `--password-stdin`，然后把密码写进子进程的 stdin。这在答辩时是个明确的安全性加分点。

进度是一行一个 JSON

核心每处理一批就往 stdout 打一行完整 JSON 并立刻刷新。界面开一个线程逐行读、逐行解析。不要等进程结束再读，那样进度条会一直不动然后一次跳完。

退出码 1 不是失败

1 表示“包做出来了，但有文件被跳过”（比如被别的程序占着）。界面要显示成黄色的“完成，N 项被跳过”并能展开看清单，而不是红色的失败。

界面侧大致长这样

```
proc = subprocess.Popen(
    [CBK, "backup", "--source", src, "--dest", dst,
      "--compress", "huffman", "--encrypt", "aes128-cbc",
      "--password-stdin", "--progress", "json"],
    stdin=PIPE, stdout=PIPE, text=True, encoding="utf-8")

proc.stdin.write(password + "\n"); proc.stdin.flush()

for line in proc.stdout:          # 逐行读，每行一个事件
    ev = json.loads(line)
    if ev["type"] == "progress": update_bar(ev["done"], ev["total"])
    elif ev["type"] == "warn":  append_warning(ev["path"], ev["msg"])
    elif ev["type"] == "result": show_result(ev)
```

// 核心吐出来的事件长这样，每行一个完整 JSON

```
{ "type": "start",    "op": "backup", "total": 4567, "totalBytes": 1234567890 }
{ "type": "progress", "done": 123, "bytes": 45678901, "current": "src\\main.cpp" }
{ "type": "warn",     "path": "C:\\x\\y.dat", "code": "E_SHARING_VIOLATION", "msg": "文件被占用，已跳过" }
{ "type": "result",   "status": "partial", "entries": 4566, "skipped": 1, "elapsedMs": 8421 }
```

05 打包和压缩要实现的两个接口 A 重点

A 的两块工作对应两个不同层次的接口：**打包决定"条目边界在哪"**（结构层），**压缩决定"字节怎么变换"**（字节层）。写实现类往注册表里一注册就行，不需要改 Z 的任何文件。

接口一：IPacker（打包解包）

Z 的 Scanner 会一条一条地把文件交给你，每条带一个 `EntryMeta`（路径、类型、时间戳、权限等）和一段内容。你的任务是决定这些东西按什么格式排进字节流，以及怎么再解析回来。

```

class IPacker {
public:
    virtual std::string name() const = 0;    // "tar" / "cpio"

    // — 打包: Scanner 每给一条就调一轮 —
    virtual void beginEntry(const EntryMeta& meta, ISink& out) = 0;
    virtual void writeData(const uint8_t* data, size_t len, ISink& out) = 0;
    virtual void endEntry(ISink& out) = 0;
    virtual void finish(ISink& out) = 0;      // 写结束标记

    // — 解包: 顺序读, 解析出一条就回调一次 —
    virtual void unpack(ISource& src,
        const std::function<void(const EntryMeta&)>& onEntry,
        const std::function<void(const uint8_t*, size_t)>& onData) = 0;
};

```

建议算法一：POSIX ustar (tar)

512 字节块对齐，头部带校验和。有公开标准可查，而且产物能用 7-Zip 直接打开验证——演示时这一点很好用。Windows 特有的权限串和属性位放进 PAX 扩展头。

建议算法二：SVR4 cpio (newc)

不做 512 对齐，头部是 ASCII 十六进制。跟 tar 形成对照，答辩时可以直接讲“两种打包格式在小文件多的场景下的空间开销差多少”——这是现成的一页 PPT。

Z 会先写一个最简单的 NativePacker（长度前缀 + 内容原样），一是让基础部分能先独立跑通，二是给你的 tar / cpio 做正确性对照基准：同一棵目录树用三种 packer 打包再还原，结果必须完全一致。

你产出的字节流会放进容器的哪一段



文件头和管线描述必须明文：还原时要先读出"这个包用了哪个 packer、哪种压缩、哪种加密"，才能组装出正确的处理链。如果这段也加密了就成了死锁。这也意味着加密只保护数据内容，不隐藏"用了哪种加密"这个事实——所有加密归档格式都是这么做的。

接口二：IStage（压缩解压）

比 IPacker 简单得多——它不关心条目边界，只是一个"进去一堆字节，出来另一堆字节"的变换器。加密也实现同一个接口，所以你和 B 写的东西可以任意串联。

```
class IStage {
public:
    virtual std::string name() const = 0;    // "huffman" / "lz77"

    // 处理一块输入，产出写进 out。允许内部缓冲，不要求 1:1。
    virtual void process(const uint8_t* data, size_t len, ISink& out) = 0;
    // 冲刷缓冲、写尾部（比如 Huffman 的位对齐填充）
    virtual void finish(ISink& out) = 0;
};

// 每个 Stage 都必须提供配对的逆变换
class IStageFactory {
public:
    virtual std::unique_ptr<IStage> createForward() = 0;    // 压缩
    virtual std::unique_ptr<IStage> createInverse() = 0;    // 解压
};
```

每个 Stage 必须自带一个往返测试：随机生成 100 组数据（含全零、全 0xFF、纯文本、已压缩数据这些边界情况），跑 `forward` 再 `inverse`，断言和原文一字节不差。这个测试要进 CI，一挂就阻止合并。压缩算法的 bug 最坑的地方在于：备份的时候一切正常，还原的时候才发现数据全毁了。

加密用的是上一节那个 `IStage`，接口完全一样，只多一个拿密码的方法。

```
// IStage 上给加密用的可选方法
virtual void setPassword(const std::string& pwd) {}
```

先做 XOR / 维吉尼亚

十几行就能写完，目的是尽早把"界面 → 核心 → 加密 → 还原"这条完整链路跑通。链路通了再换真算法，比一上来就啃 AES 顺得多。

建议第 6 周前完成

再做 AES-128-CBC

S 盒、密钥扩展、CBC 模式全部手写。IV 随机生成并写在你这段输出的开头，解密时先读出来。尾部用 PKCS#7 补齐。

建议第 8 周前完成

建议加一个 PBKDF2-HMAC-SHA256 做密钥派生。不要把用户输入的密码直接当密钥用——用户密码可能只有 6 个字符，熵远远不够。PBKDF2 迭代一万次把它拉伸成 128 位密钥，大约 150 行代码。这是答辩时"你们的加密有什么考虑"这个问题的标准答案，性价比很高。

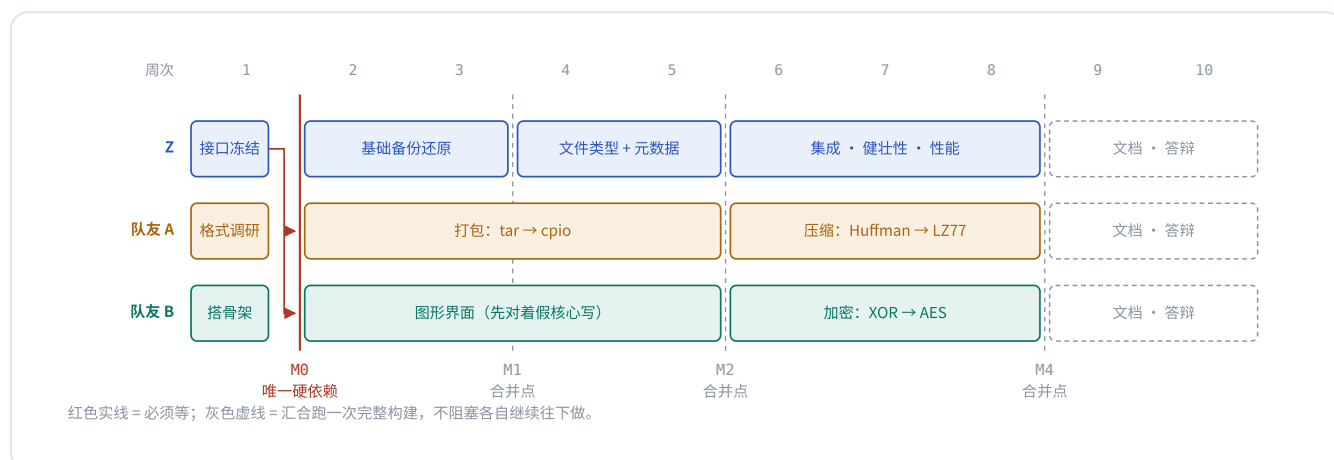
一个容易踩的坑

你的 Stage 是流式的，会被喂很多次 `process`，每次的长度不保证是块大小的整数倍。

CBC 模式必须自己维护一个不满 16 字节的余料缓冲，攒够一块再加密，最后在 `finish` 里补齐并冲刷。直接假设"每次进来都是 16 的倍数"，在小文件多的目录上必挂。

先说清楚一个容易误会的地方：**里程碑是合并点，不是关卡**。不存在"三个人都做完 M1 才能开始 M2"这回事。整个项目只有一处真正的硬依赖，在**第一周**；过了那一周，三条

线各走各的，只在几个日期上汇合、跑一次完整构建。



整张图上只有两条红色箭头是"必须等": Z 在第一周把三个接口头文件合进 main, A 和 B 才知道自己要实现什么函数。之后三条线是真正并行的——A 写 tar 的时候不需要 Z 的 Scanner 写完, B 做界面的时候不需要 `cbk.exe` 存在。

为什么能并行：三个不用等人的办法

并行不是靠自觉，是靠每个人手上都有办法把自己的东西单独跑起来。这三招是这份方案能并行的技术前提：

A 测打包：自己捏 EntryMeta

不需要 Z 的 Scanner。测试里手写几个 `EntryMeta`（一个普通文件、一个目录、一个符号链接），塞给你的 packer，再 `unpack` 回来比对。这就是完整的正确性测试，跟 Z 的进度完全无关。

压缩和加密测试：一堆随机字节

`IStage` 的输入输出就是裸字节，跟这个项目的其它部分零耦合。造一段随机数据，`forward` 再 `inverse`，断言完全一致。你可以在一个空白项目里把整个 Huffman 或 AES 写完测完，再搬进仓库。

B 做界面：写一个假的 `cbk.exe`

二十行 Python 就能伪造一个核心：睡一会儿，往 stdout 打几行符合协议的 JSON，最后返回一个退出码。界面对着它开发，进度条、警告列表、错误处理全都能做完、能演示。等 Z 的真核心出来，把路径一换就行。

```
# tools/fake_cbk.py — B 第一周就该有的东西，跟真核心的协议一模一样
import sys, json, time
print(json.dumps({"type": "start", "op": "backup", "total": 200}), flush=True)
for i in range(1, 201):
    time.sleep(0.02)
    print(json.dumps({"type": "progress", "done": i, "current": f"src/file_{i}.cpp"}), flush=True)
    if i == 137:
        print(json.dumps({"type": "warn", "path": "C:\\\\locked.dat",
                          "code": "E_SHARING_VIOLATION", "msg": "文件被占用，已跳过"}), flush=True)
print(json.dumps({"type": "result", "status": "partial", "entries": 199, "skipped": 1}), flush=True)
sys.exit(1) # 1 = 部分成功，正好用来调黄色的"完成，1 项被跳过"
```

注意那个 `flush=True`。它同时是假核心和真核心都必须做的事：不逐行刷新，输出会攒在管道缓冲里，界面的进度条会一直不动然后在最后一秒跳到 100%。B 用这个假核心开发，等于第一周就把这个坑替真核心踩掉了。

那合并点是干什么的

合并点存在，是因为**并行开发真正的风险不是"等人"，是"八周不碰头，第九周发现三个人的东西根本接不上"**。所以每隔两三周强制汇合一次：三条分支合进 `main`，跑一次完整的备份还原，打一个 git tag。接口理解有偏差、谁的假设错了，在这时候暴露只需要改两天；拖到第九周暴露，就是通宵。

合并点**不要求每个人都做完当期的活**。A 的 `cpio` 没写完，M2 照样合——合进去的是当时能跑的 `tar`，`cpio` 下个合并点再进。唯一的要求是：合进 `main` 的东西必须能构建、能跑通测试。没写完的留在自己分支上。

每个合并点各自交什么

第 1 周

M0 • 接口冻结

- Z** 把 `IPacker` / `IStage` / `EntryMeta` 三个头文件写完并合进 `main`；搭 CMake 骨架和 CI
- A** 读 `tar` 格式标准，把测试用例先写出来（对着头文件写，不用等实现）
- B** 搭 PySide6 骨架，把 `core_bridge.py` 的 `subprocess` 封装先写好

第 2–3 周

M1 • 基础链路跑通 v0.1-base

- Z** 备份还原打通（`NativePacker`，不压不加密）
- A** `tar packer` 开工
- B** 界面能调 `cbk backup` 并正确显示进度条

第 4–5 周

M2 • 类型与元数据 v0.3-metadata

- Z 符号链接、硬链接、目录联接；属性位、时间戳、属主、权限
- A tar 完成并通过往返测试
- B 界面完善：还原、浏览包内容、警告清单

第 6–7 周

M3 • 压缩接入 v0.5-compress

- A cpio + Huffman
- B XOR 加密，跑通完整链路
- Z 协助集成，补集成测试

第 8 周

M4 • 加密接入 v0.6-crypt

- A LZ77
- B AES + PBKDF2
- Z 性能与健壮性：大文件、一万个小文件、长路径

第 9 周

M5 • 集成与文档 v1.0-gui

- 全员 三份文档定稿：需求分析说明书、系统设计文档、软件测试报告

第 10 周

M6 • 答辩

- 全员 PPT、演示彩排、在助教机器上验证一遍能跑

如果有人掉队了会怎样

这套结构的一个好处是**风险是隔离的**。A 的 LZ77 到第八周还没写完，不影响 Z 和 B 的任何工作，产品照常能交，只是少一种压缩算法、少 10 分。B 的 AES 卡住了，把加密选项在界面上灰掉就行，其它功能一样能演示。

唯一一个"卡住就全组停摆"的位置是 M0。所以这套排期里真正需要盯的只有第一周——之后每个人的进度只对自己那部分分数负责，不会互相拖累。

M0 是整个排期里唯一不能拖的一周。接口头文件不定下来，A 和 B 就只能干等 Z，三个人变成串行。宁可第一周花两天把接口反复推敲清楚，也不要到第六周为了塞一个新算法回头改容器格式——那时候改一次，三个人的代码全要动。

08 协作规矩

评分表里"版本管理工具使用情况""人员分工安排情况""项目进度把控情况"都是**过程性**证据，最后一周突击补不出来。所以从第一天就按规矩来，成本很低。

分支

`main` 受保护，只接受 Pull Request，且必须 CI 通过。

- `feat/core-*` — Z
- `feat/pack-*`、`feat/compress-*` — A
- `feat/crypt-*`、`feat/gui` — B

提交信息

用 Conventional Commits，格式是 类型(范围)：说明：

- `feat(pack)`：实现 `ustar` 头部写入
- `fix(crypt)`：修正 CBC 余料未冲刷
- `test(stage)`：补 Huffman 往返 20 例

好处是 `git log` 直接能生成答辩用的进度图。

CI

GitHub Actions 在 Windows 上跑构建 + 测试 + `cpplint`。一个配置文件同时兑现评分表里的"能否自动化构建"和"版本管理工具使用情况"两项，性价比最高的一件事。

代码规范

仓库根目录有 `.clang-format`，提交前跑一次 `scripts/format.ps1`。评分表另有"注释达到 10% / 20%"两档，用 Doxygen 风格写在头文件里，`scripts/comment_ratio.ps1` 可以随时统计——别等到最后一周才发现不够。

文档同步写

系统设计文档里的构件图、类图、顺序图，在 M0 和 M2 各出一版，跟代码一起演进。等第九周才开始画，画出来的一定和代码对不上，而"用例图和用例描述一致程度"是明确的评分项。

09 需要你们确认的事

读到这里就差不多了。下面几条请逐条回复——有异议现在提，接口一冻结再改成本就高了。

01 上面的分工和时间线，认不认？

尤其是 M0 那一周的三件事，能不能在一周内交。如果谁最近有别的事顶不上，现在说，排期还能调。

02 A

打包做 tar + cpio 两种，可以吗？

每种算法 10 分，做两种是 20 分。如果你想自研一种格式也行，但要能说清楚它相对 tar 的取舍在哪，否则答辩时不好讲。

03 A

压缩做 Huffman + LZ77 两种，可以吗？

这两个组合起来就接近 Deflate 的原理，答辩时可以讲"两种思路各自压什么样的数据更有效"，是一页很实的 PPT。

04 B

加密做 XOR + AES-128-CBC，要不要加 PBKDF2？

PBKDF2 大约多 150 行，但它是"你们的加密设计有什么考虑"这个问题的现成答案。建议做。

05 B

界面用 PySide6 还是 PyQt6？

建议 PySide6，LGPL 授权，商用和分发都没有麻烦。两者 API 几乎一样。

06 仓库放 GitHub 还是 Gitee？

建议 GitHub 公开仓库——公开仓库的 Actions 不限量，私有仓库的 Windows runner 是按 1:2 计费的，额度撑不了一学期。

07 项目管理工具用哪个，谁来维护？

评分表点名了 Teambition，GitHub Projects 也行。关键是要有能截图的证据：任务卡、负责人、截止日期。需求分析文档里要的甘特图也从这里导出。

08 AI 课程证书，这周就去拿。

整整 10 分，是全项目里最容易拿的分，三个人各自去做完就行。别拖到最后。

确认完这几条，Z 会把接口头文件提成 PR 发到群里，两位过一遍就可以各自开工了。

CBK · 组内开工说明 v0.1 · 2026-08-31 · 有异议直接在群里说，改完发 v0.2